

DOSSIER PROJET

DIGITALISATION DU SAV



Projet présenté par : FRED SAMMY GOULJIAR

Entreprise : Groupe RAVATE – Île de La Réunion

Juillet 2026

***EN PREPARATION DU TITRE PROFESSIONNEL DE DEVELOPPEUR WEB ET
WEB MOBILE***

TABLE DES MATIERES

1. INTRODUCTION – CONTEXTE DU PROJET	3
2. LISTE DES COMPÉTENCES COUVERTES PAR LE PROJET	4
3. RÉSUMÉ DU PROJET	6
4. SUIVI DU PROJET	7
A. Organisation et cadrage	7
B. Réalisation des maquettes	10
C. Développement back-end	11
C1 : Connexion à la base de donnée	13
C2. Connexion à l'API JIRA	14
C3. Services	16
C4. Contrôleurs	17
C4. Routes	18
C5. Middleware	19
D. Développement front-end	21
E. Versionning et gestion de code (GIT / GitHub)	22
5. CAHIER DES CHARGES	23
A. Besoins identifiés	23
B. Objectifs	23
C. Cibles et utilisateurs	23
D. Users Stories	24
E. MVP (Produit Minimum Viable)	30
F. Évolutions potentielles	30
6. WIREFRAMES ET MAQUETTES	31
7. SPÉCIFICATIONS TECHNIQUES	36
A. Technologies utilisées	36
B. Navigateurs compatibles	37
C. Déploiement envisagé	37
D. Base de données	37
E. Routes	41
F. API JIRA	41
8. ARCHITECTURE	42
9. SÉCURITÉ ET VEILLE TECHNOLOGIQUE	43
10. LIVRABLE	45
11. JEUX d'ESSAI	46
12. CONCLUSION	50

1. INTRODUCTION – CONTEXTE DU PROJET

Titulaire d'un Diplôme d'Études Comptables et Financières (DECF), j'ai intégré il y a 17 ans le Groupe RAVATE (Weldom, GIFI, Leroy Merlin, GIFI, INTERSPORT, MAISON DU MONDE...), entreprise située sur l'île de La Réunion.

Après avoir occupé le poste de contrôleur de gestion pendant 15 ans, j'ai rejoint depuis deux ans l'équipe MOA (Maîtrise d'Ouvrage Applicative), en charge du déploiement du nouvel ERP Sage X3 au sein des différentes enseignes du groupe.

Mon rôle actuel consiste à accompagner les équipes dans la conduite du changement, tout en pilotant des projets applicatifs. Ce poste me permet d'être au plus près des utilisateurs et de comprendre leurs problématiques quotidiennes, afin d'apporter des solutions adaptées et efficaces.

Passionné d'informatique, j'ai souvent conçu des outils internes pour faciliter le travail des équipes. N'ayant pas de bases officielles en programmation, j'ai choisi de suivre la formation Développeur Web et Web Mobile afin de consolider mes acquis et préparer la présentation du Titre Professionnel DWWM.

Le projet de digitalisation du SAV est né d'une observation simple : les équipes utilisaient encore Excel pour gérer et imprimer les fiches SAV, entraînant perte de temps, doublons et manque de traçabilité. L'idée de créer une application web dédiée est alors devenue une évidence, pour moderniser et fluidifier ce processus.

2. LISTE DES COMPÉTENCES COUVERTES PAR LE PROJET

Conception d'une application web complète (front & back)

Le projet a permis de concevoir une application web complète en architecture MVC, alliant une partie front-end en React et un back-end développé sous Node.js avec le framework Express.

Cette approche modulaire a facilité la séparation des responsabilités entre interface, logique métier et accès aux données.

Création d'une base de données relationnelle MySQL

La conception de la base de données a suivi une approche normalisée (MCD, MLD, dictionnaire des données).

Les relations entre tables ont été définies pour éviter les redondances et assurer l'intégrité référentielle.

Un environnement de test hébergé sur AlwaysData a permis de valider les requêtes SQL et de simuler les opérations CRUD.

Des jeux de données factices ("bouchonnées") ont servi à tester la robustesse et la cohérence du modèle. (cf 12. JEUX d'ESSAI)

Développement d'une API REST sécurisée (Express / Node.js)

Le back-end repose sur Express, le framework de Javascript, permettant de créer une API REST structurée et maintenable.

Les différentes routes (authentification, fiches, clients, tickets Jira...) ont été testées.

Des middlewares ont été ajoutés pour l'authentification, la sauvegarde des fichiers (photos, signatures...)

les API assurent la liaison entre la base de données MySQL et l'interface React.

Développement d'un frontend dynamique (React)

La partie front-end a été développée avec **React**, afin de bénéficier d'une interface moderne, fluide et réactive. Certains composants ont été pensés pour être réutilisable (signatures, impression pdf etc.). Une attention particulière a été portée à la responsivité pour un affichage optimal sur PC, tablette et smartphone. Il s'agissait ici de rendre l'application totalement utilisable dans l'entrepôt.

Authentification sécurisée et gestion des sessions

L'application intègre un système d'authentification sécurisé basé sur JWT (JSON Web Token).

Les mots de passe sont chiffrés via bcryptjs avant d'être enregistrés en base, garantissant la confidentialité des comptes.

Les routes protégées nécessitent un token valide, évitant tout accès non autorisé.

Ce mécanisme assure la sécurité des données utilisateurs et la traçabilité des actions effectuées.

Génération de documents PDF et gestion de signatures électroniques

Les fiches SAV peuvent être exportées au format PDF via la librairie html2pdf.js.

Chaque document inclut les informations client, produit, signatures électroniques et photos associées.

Les signatures du client et du Responsable SAV sont capturées et sauvegardées sous forme d'images.

Ce système permet de dématérialiser totalement le processus tout en assurant la validité des documents générés.

Gestion des fichiers et intégration d'images

Le projet gère l'envoi, le stockage et l'affichage d'images, notamment les photos des produits et signatures.

Les fichiers sont enregistrés dans un répertoire sécurisé du serveur (/uploads/signatureFournisseurs et /uploads/signature)

Des contrôles ont été mis en place sur la taille, le type et le nombre d'images (3 maximum par fiche).

Cette fonctionnalité contribue à enrichir la traçabilité du SAV et à conserver une preuve visuelle des produits traités.

3. RÉSUMÉ DU PROJET

Le projet consiste à digitaliser la gestion des fiches SAV, actuellement gérées via des fichiers Excel imprimés.

Cette application web permettra :

- De récupérer les tickets Jira d'un Kanban et d'un composant précis.
- De créer, consulter, modifier et imprimer les fiches SAV.
- D'ajouter jusqu'à 3 photos produits.
- D'associer chaque fiche à un client existant ou nouvellement créé.
- De gérer les doublons clients (email).
- D'intégrer la signature électronique du client et du Responsable SAV.
- De générer un PDF complet pour archivage ou envoi.
- D'envoyer des emails automatiques de suivi et de relance.
- De présenter un tableau de bord avec à terme une intégration Power BI.

Objectifs :

- Gain de temps pour les équipes SAV.
- Meilleure traçabilité et réduction des erreurs.
- Interface moderne, mobile et sécurisée.

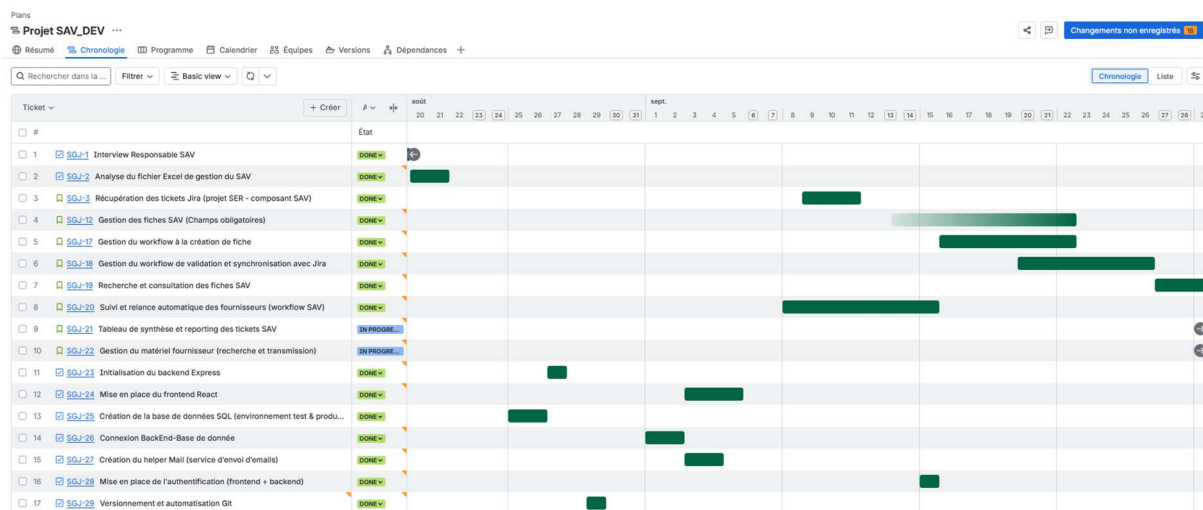
4. SUIVI DU PROJET

A. ORGANISATION ET CADRAGE

- Interviews utilisateurs & User Stories**

Avant le développement, plusieurs entretiens ont été menés avec la Responsable SAV pour définir les besoins.

Un Kanban Jira a été mis en place pour suivre les tâches.



Analyse du fichier Excel :

Faiblesses d'un fichier Excel comme application

1. Fiabilité limitée

- Risque élevé d'erreurs manuelles (formules écrasées, données supprimées, doublons).

2. Difficulté de collaboration

- Fichier souvent partagé par email ou sur un réseau → versions multiples, conflits d'enregistrement.
- Aucun suivi des modifications ni traçabilité fiable.

3. Sécurité insuffisante

- Pas de mot de passe d'authentification
- Pas de gestion d'utilisateurs

4. Performances et évolutivité

- Devient lent et instable avec de gros volumes de données.
- Aucune réelle base de données relationnelle en arrière-plan.

5. Maintenabilité et évolutions compliquées

- Les macros et formules sont souvent dépendantes d'un utilisateur ou d'un poste.
- Difficulté à faire évoluer la structure sans casser des liens ou des formules.

6. Visualisation et automatisation limitées

- Les rapports et tableaux de bord nécessitent des manipulations manuelles.
- Aucune logique métier automatisée ni notifications ou intégrations externes.

Intérêt de passer à une application web dynamique, sécurisée et performante

1. Fiabilité et cohérence des données

- Base de données centralisée (MySQL) garantissant l'intégrité des données.
- Validation automatique des champs et règles métier.

2. Travail collaboratif en temps réel

- Plusieurs utilisateurs peuvent travailler simultanément sans conflit.
- Historique, traçabilité et gestion des rôles intégrées.

3. Sécurité renforcée

- Authentification sécurisée (login/mot de passe, JWT).
- Données protégées côté serveur et sauvegardes automatisées.

4. Performance et évolutivité

- Conçue pour gérer de gros volumes et de nombreux utilisateurs.

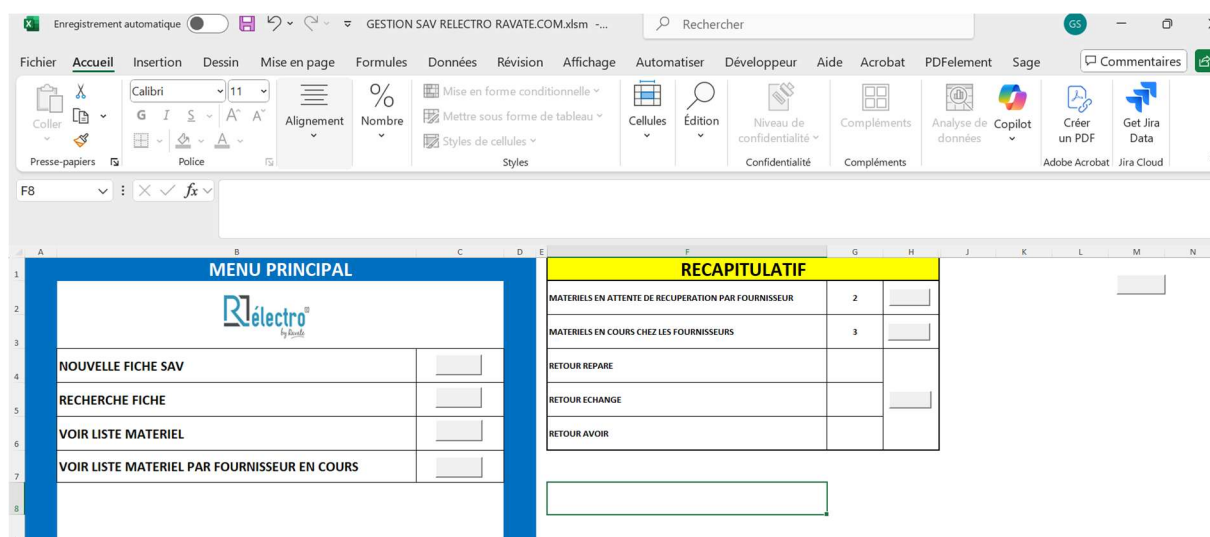
5. Tableaux de bord et automatisation

- Visualisation dynamique, filtres, graphiques interactifs. (mettre en place avec Power BI)
- Automatisation des calculs, alertes et envois d'emails.

6. Accessibilité

- Accessible depuis n'importe quel appareil via un navigateur.
- Compatible avec les outils modernes (Power BI).

ci-dessous quelques captures du fichier Excel



CREATION FICHE SAV			
Date	09/10/2025		
CLIENT			
NOM			
PRENOM			
TEL			
ADRESSE			
VILLE			
EMAIL			
REF		EAN	
DESIGNATION			
VALEUR ACHAT		RAY	35804264
NUMERO DE SERIE		N° SER	75896
DATE D'ACHAT		N° CMD	150266
PANNE			
ETAT GENERAL MATE			
ACCESSOIRES			
OBSERVATIONS			
ENREGISTRER		ANNULATION	

RETOUR		MAJ		RESULTATS RECHERCHE				
Num Fiche	DATE Prise en charge	CLIENT	TEL 1	TEL 2	Adresse	Ville		
NUM FICHE								
	458424							
NOM CLIENT								
PRENOM								
TEL								
RECHERCHE								

B. REALISATION DES MAQUETTES

Les maquettes de l'application ont été conçues à l'aide de **Figma**, en suivant une approche centrée utilisateur (UX/UI design).

Elles ont permis de valider la cohérence visuelle et fonctionnelle avant le développement front-end.

Deux types de pages ont été définis :

- **Pages imposées :**

Les écrans "**Synthèse**" et "**Ticket Jira**" ont été fournis dans le cadre du projet, avec une charte graphique et une structure déjà définies.

Leur intégration visait à garantir la conformité avec les outils existants du groupe (notamment Jira).

- **Pages à conception libre :**

Pour toutes les autres pages (connexion, tableau de bord, gestion des fiches, clients, signatures, etc.), j'ai eu carte blanche sur la conception, tout en respectant :

- les conventions UX/UI standard du web (hiérarchie visuelle, contraste, affordance, cohérence des icônes, responsive design),
- la charte graphique de l'entreprise (sobriété, lisibilité),
- et les bonnes pratiques d'ergonomie (zones de clics, navigation claire).

C. DEVELOPPEMENT BACK-END

- API REST développée avec **Express**.
- Sécurisation des routes avec **JWT** et **bcryptjs**.
- Connexion MySQL direct

```
PS C:\Users\Sammy> cd projet\SAV_dev
PS C:\Users\Sammy\projet\SAV_dev> mkdir backend
```

Puis : cd backend

Initialiser un projet Node

npm init -y

puis installation des packages :

Express : npm install express

Dotenv : npm install dotenv

Mysql2 : npm install mysql2

bcryptjs, multer, node-mailjet, netc....

Le projet utilise le système de modules *CommonJS* pour l'importation des fichiers

Dans mon package.json : `"type": "commonjs",`

Création de la base de donnée



Création de la table client :

```
CREATE TABLE `client` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `nom` varchar(100) NOT NULL,
  `prenom` varchar(100) NOT NULL,
  `email` varchar(150) NOT NULL,
  `adresse` varchar(255) DEFAULT NULL,
  `code_postal_id` varchar(10) NOT NULL,
  `tel1` varchar(10) DEFAULT NULL,
  `tel2` varchar(10) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `email` (`email`),
  KEY `fk_client_code_postal` (`code_postal_id`),
  CONSTRAINT `fk_client_code_postal` FOREIGN KEY (`code_postal_id`) REFERENCES `code_postal` (`code_postal`) ON UPDATE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=48 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Création de la table fiche

```
CREATE TABLE `fiche` (
  `id_fiche` int(11) NOT NULL AUTO_INCREMENT,
  `code_rav` varchar(8) DEFAULT NULL,
  `numero_serie` varchar(100) DEFAULT NULL,
  `date_achat` date DEFAULT NULL,
  `Num_CMD` varchar(100) DEFAULT NULL,
  `Num_Ser` varchar(100) DEFAULT NULL,
  `Panne` text DEFAULT NULL,
  `etat_general` text DEFAULT NULL,
  `accessoires` text DEFAULT NULL,
  `observations` text DEFAULT NULL,
  `date_creation` date NOT NULL DEFAULT curdate(),
  `nom` varchar(100) DEFAULT NULL,
  `prenom` varchar(100) DEFAULT NULL,
  `tel1` varchar(20) DEFAULT NULL,
  `tel2` varchar(20) DEFAULT NULL,
  `adresse` varchar(100) DEFAULT NULL,
  `ville` varchar(20) DEFAULT NULL,
  `email` varchar(100) DEFAULT NULL,
  `photo` varchar(200) DEFAULT NULL,
  `designation` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`id_fiche`)
) ENGINE=InnoDB AUTO_INCREMENT=27 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Ces script définissent notamment :

- les tables utilisées pour stocker les données,
- les clés primaires permettant d'identifier chaque enregistrement,
- les clés étrangères assurant l'intégrité des relations entre les tables,
- les contraintes garantissant la cohérence des données.

C1 : CONNEXION A LA BASE DE DONNEE

Création d'un dossier config et d'un fichier db.js

```
const mysql = require('mysql2');
const sql = require('mssql'); //
require('dotenv').config();

// --- Connexion MySQL ---
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
  timezone: 'Z', // UTC
  dateStrings: true // IMPORTANT
});

const db = pool.promise();
```

Connexion à la base de données MySQL

La connexion à la base de données est gérée via la librairie mysql2 en utilisant un pool de connexions.

Les paramètres sensibles (hôte, identifiants, base, port) sont stockés dans un fichier .env et chargés via dotenv, afin de garantir la sécurité et la flexibilité selon les environnements.

Le pool est configuré pour accepter jusqu'à 10 connexions simultanées. Lorsque toutes les connexions sont occupées, les requêtes supplémentaires sont mises en attente, ce qui permet de contrôler la charge et d'assurer la stabilité de l'application.

Le fuseau horaire est fixé à UTC (timezone: 'Z') afin d'éviter tout décalage horaire, et les dates sont retournées sous forme de chaînes de caractères (dateStrings: true) pour éviter les conversions automatiques côté JavaScript.

Une requête simple (SELECT 1) est exécutée au démarrage de l'application afin de vérifier la disponibilité et la bonne configuration de la base de données.

Les requêtes sont ensuite exécutées via une interface Promise, permettant l'utilisation de async / await.

C2. CONNEXION A L'API JIRA

```
// jira
const { JIRA_EMAIL, JIRA_API_TOKEN, JIRA_BASE_URL } = process.env;
const { db } = require("../config/db");

if (!JIRA_EMAIL || !JIRA_API_TOKEN || !JIRA_BASE_URL) {
  throw new Error(
    " JIRA_EMAIL, JIRA_API_TOKEN et JIRA_BASE_URL doivent être définis dans .env"
  );
}

// Supprime le / en fin d'url
const BASE = `${JIRA_BASE_URL.replace(/\/+$/, "")}/rest/api/3`;

// Authentification basique avec email + token encodage buffer binaire + base 64 et ajoute prefixe basic http
function authHeader() {
  const b64 = Buffer.from(`${JIRA_EMAIL}:${JIRA_API_TOKEN}`).toString("base64");
  return `Basic ${b64}`;
}

// Requête générique vers Jira
async function jiraFetch(path, opts = {}) {
  const res = await fetch(`${BASE}${path}`, {
    method: opts.method || "GET",
    headers: {
      Accept: "application/json",
      Authorization: authHeader(),
      ...(opts.body ? { "Content-Type": "application/json" } : {}),
      ...(opts.headers || {}),
    },
    body: opts.body
      ? typeof opts.body === "string"
        ? opts.body
        : JSON.stringify(opts.body)
      : undefined,
  });

  if (!res.ok) {
    const text = await res.text().catch(() => "");
    const err = new Error(
      `Jira ${res.status} ${res.statusText} - ${text.slice(0, 400)}`
    );
    err.status = res.status;
    err.details = text;
    throw err;
  }

  if (res.status === 204) return null; // Jira renvoie parfois 204 sans body
  return res.json();
}
```

L'intégration avec l'API Jira repose sur l'utilisation d'un token d'authentification généré depuis la plateforme Atlassian de l'entreprise.

Ce token, associé à une adresse email, permet une authentification sécurisée auprès de l'API Jira (valable pour une durée d'un an).

Les paramètres nécessaires (JIRA_EMAIL, JIRA_API_TOKEN, JIRA_BASE_URL) sont stockés dans un fichier .env.

Une vérification est effectuée au démarrage de l'application afin de s'assurer que toutes les variables requises sont définies. En cas d'absence, l'application est volontairement interrompue afin d'éviter tout comportement incorrect.

L'URL de base de l'API Jira est normalisée afin de supprimer les éventuels slashes en fin d'URL, puis complétée avec le chemin de l'API REST v3.

L'authentification est réalisée via le schéma Basic Authentication.

Les identifiants (email:token) sont encodés en Base64 à l'aide de `Buffer.from(...).toString('base64')`, puis transmis dans l'en-tête HTTP Authorization.

Une fonction générique est mise en place pour centraliser les appels vers l'API Jira. Elle permet :

- de définir la méthode HTTP (GET, POST, etc.)
- de gérer les headers communs
- d'envoyer des données au format JSON
- de gérer les erreurs HTTP et les réponses de l'API de manière uniforme

Toutes les réponses sont échangées au format JSON, conformément aux standards de l'API Jira.

C3. SERVICES

```
const { db } = require('../config/db');
const bcrypt = require('bcryptjs');

async function addUser(email, mot_passe, prenom, nom) {
  try {
    const hash = await bcrypt.hash(mot_passe, 10); // 10 = saltRounds

    const sql = `INSERT INTO users (email, mot_passe, prenom, nom) VALUES (?, ?, ?, ?)`;
    const [result] = await db.query(sql, [email, hash, prenom, nom]);

    return { id: result.insertId, email, prenom, nom };
  } catch (err) {
    throw err;
  }
}

async function loginUser(email, mot_passe) {
  try {
    // Récupérer l'utilisateur
    const sql = `SELECT * FROM users WHERE email = ? LIMIT 1`;
    const [rows] = await db.execute(sql, [email]);

    if (rows.length === 0) {
      return null; // email inexistant
    }

    const user = rows[0];

    // Vérifier le mot de passe
    const isMatch = await bcrypt.compare(mot_passe, user.mot_passe);
    if (!isMatch) {
      return null; // mot de passe incorrect
    }

    return user;
  } catch (err) {
    console.error("Erreur lors du login :", err);
    throw err;
  }
}

module.exports = { addUser, loginUser };
```

Service d'authentification utilisateur (usersServices)

Le service usersServices gère la logique métier liée aux utilisateurs, notamment l'authentification.

Une fonction asynchrone est définie afin de gérer la connexion d'un utilisateur. Elle reçoit en paramètres l'adresse email et le mot de passe saisis par l'utilisateur.

Une requête SQL préparée est exécutée afin de récupérer les informations de l'utilisateur correspondant à l'email fourni.

L'utilisation des paramètres (?) permet d'échapper automatiquement les valeurs et de se protéger contre les injections SQL.

Si aucun utilisateur n'est trouvé, la fonction retourne null. (voir message dans le contrôleur)

Si un utilisateur est trouvé, le mot de passe saisi est comparé au mot de passe hashé stocké en base de données à l'aide de la librairie bcryptjs.
En cas de non-correspondance, la fonction retourne également null.

Si l'authentification est valide, les informations de l'utilisateur sont retournées au contrôleur, qui se charge de la suite du traitement (session, token, redirection, etc.).

Les erreurs techniques sont capturées et remontées afin d'être correctement gérées au niveau supérieur.

C4.CONTROLLEURS

```
const userService = require('../../services/Auth/usersServices');
const jwt = require('jsonwebtoken');

const userController = {

  // Connexion utilisateur
  login: async (req, res) => {
    try {
      const { email, mot_passe } = req.body;

      if (!email || !mot_passe) {
        return res.status(400).json({ message: "Email et mot de passe obligatoires" });
      }

      const user = await userService.loginUser(email, mot_passe);
      if (!user) {
        return res.status(401).json({ message: "Email ou mot de passe incorrect" });
      }

      // Générer un JWT (contient id, email, prenom)
      const token = jwt.sign(
        { id: user.id, email: user.email, prenom: user.prenom },
        process.env.JWT_SECRET,
        { expiresIn: "24h" }
      );

      // Réponse uniforme : message + token + user
      res.json({
        message: "Connexion réussie",
        token,
        user: {
          id: user.id,
          email: user.email,
          prenom: user.prenom,
          nom: user.nom,
          role: user.role
        }
      });
    } catch (error) {
      console.error("Erreur login :", error);
      res.status(500).json({ message: "Erreur serveur" });
    }
  }
};

module.exports = userController;
```

Le contrôleur d'authentification est responsable de la logique métier liée à la connexion des utilisateurs ainsi que de la gestion des réponses HTTP.

- la librairie jsonwebtoken, utilisée pour la gestion des tokens JWT

Une fonction asynchrone login est définie afin de traiter les requêtes de connexion. Elle récupère les informations envoyées dans le corps de la requête (req.body), notamment l'adresse email et le mot de passe.

Une première validation est effectuée afin de vérifier la présence des champs requis. En cas d'oubli, une réponse d'erreur est immédiatement renvoyée.

Le contrôleur appelle ensuite le service d'authentification afin de vérifier les identifiants.

Si l'email ou le mot de passe est incorrect, une réponse d'erreur est retournée sans préciser lequel des deux est invalide, conformément aux bonnes pratiques de sécurité.

Lorsque l'authentification est validée, un token JWT est généré.

Ce token est signé à l'aide d'un secret stocké dans le fichier .env et possède une durée de validité de 24 heures.

Enfin, une réponse JSON est renvoyée au client contenant :

- un message de succès
- le token JWT
- les informations de l'utilisateur authentifié

Les erreurs techniques sont interceptées et une réponse de type 500 – Erreur serveur est renvoyée en cas de problème inattendu.

La fonction est exportée afin d'être utilisée dans les routes Express.

C4.ROUTES

```
const express = require('express');
const router = express.Router();

const userController = require('../controller/Auth/userController');

// Insertion d'un user (POST)
router.post('/insertlogin', userController.adduser);

// Connexion (login)
router.post('/login', userController.login);

module.exports = router;
```

Ce fichier définit les routes HTTP liées aux utilisateurs, en s'appuyant sur Express et une architecture Route / Controller.

Le contrôleur utilisateur est ensuite importé. Il contient la logique métier associée à la création et à la connexion des utilisateurs.

Le router est exporté afin d'être monté dans l'application principale (index.js), ce qui permet de garder une structure modulaire et maintenable.

C5.MIDDELWARE

```
const jwt = require('jsonwebtoken');

function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization'];

  if (!authHeader) {
    console.log("pas de token")
    return res.status(401).json({ message: "Token manquant" });
  }

  const token = authHeader.split(' ')[1]; // format: "Bearer xxx"
  if (!token) {
    console.log("Token invalide")
    return res.status(401).json({ message: "Token invalide" });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // ajoute l'utilisateur décodé dans la requête
    next(); // passe au contrôleur suivant
    console.log("Token valide")
  } catch (err) {
    return res.status(403).json({ message: "Token invalide ou expiré" });
  }
}

module.exports = verifyToken;
```

Rôle du middleware verifyToken

Ce middleware a pour objectif de protéger certaines routes en vérifiant la validité d'un token JWT avant d'autoriser l'accès au contrôleur.

La librairie jsonwebtoken est utilisée pour :

- vérifier la signature du token
- décoder son contenu
- contrôler sa validité (expiration, intégrité)

Fonctionnement détaillé

1. Le middleware récupère le header HTTP Authorization
2. Authorization: Bearer <token>
3. Si le header est absent :

- la requête est bloquée
 - une réponse **401 – Token manquant** est renvoyée
4. Le token est extrait du header (Bearer xxx)
 5. Si le token est invalide ou mal formé :
 - la requête est bloquée
 - une réponse **401 – Token invalide** est renvoyée
 6. Le token est ensuite vérifié à l'aide du secret JWT stocké dans le fichier .env
 7. Si le token est valide :
 - son contenu est décodé
 - les informations de l'utilisateur sont ajoutées à l'objet req (req.user)
 - la fonction next() est appelée pour autoriser l'accès au contrôleur
 8. Si le token est expiré ou invalide :
 - une réponse **403 – Token invalide ou expiré** est renvoyée
 9. Ce middleware est ensuite appliqué uniquement aux routes nécessitant une authentification

Exemple :

```
router.get('/regroupement/statut', verifyToken, FicheController.getRegroupementByStatus);
```

Le point d'entrée du backend est le fichier index.js

D. DEVELOPPEMENT FRONT-END

- Développement React avec gestion d'état et navigation.
- Respect des bonnes pratiques UX/UI.
- Intégration de composants réutilisables.

npx create-react-app SAV_dev

dans le dossier src création des dossiers composants, pages, helpers

exemple de page :

```
import React, { useState, useEffect } from "react";
import ComposantRefresh from "../composant/fiche/Refresh";

const Synthese = () => {
  const [fiches, setFiches] = useState([]);
  const [delai, setDelai] = useState(null);
  const [sending, setSending] = useState(false);
  const [delaiPdtChezFnr, setDelaiPdtChezFnr] = useState(null);
  const [ficheId, setFicheId] = useState([]);

  const [prenom, setPrenom] = useState(""); //
  const [email, setEmail] = useState(""); //

  // --- Récup prénom utilisateur depuis localStorage
  useEffect(() => {
    const storedPrenom = localStorage.getItem("prenom");
    if (storedPrenom) {
      setPrenom(storedPrenom);
    }
  }, []);

  useEffect(() => {
    const storedEmail = localStorage.getItem("email");
    if (storedEmail) {
      setEmail(storedEmail);
    }
  }, []);

  // --- Récup stats par statut
```

Import des hooks React

useState permet de gérer l'état local du composant fonctionnel

useEffect permet d'exécuter des effets de bord :

- lecture du localStorage
- appels API

récupérer les infos de connexion depuis le localStorage

Dans l'application, les informations de session (token d'authentification) sont stockées dans le localStorage du navigateur.

Cette approche peut présenter des limites en matière de sécurité. En effet, les données stockées dans le `localStorage` sont accessibles depuis le navigateur via JavaScript et pourraient potentiellement être récupérées en cas de vulnérabilité de type XSS (Cross-Site Scripting).

Ce choix a toutefois été fait en tenant compte du contexte d'utilisation de l'application, qui est entièrement déployée en intranet. L'application n'est donc pas exposée publiquement sur Internet, ce qui réduit considérablement les risques d'attaque externe.

Par ailleurs, d'autres mécanismes de sécurité sont présents, notamment :

- l'authentification des utilisateurs,
- la gestion des accès aux routes de l'API,
- la séparation entre le frontend et le backend.

Cependant, si l'application devait évoluer vers une exposition sur Internet, cette stratégie de stockage serait réévaluée. Une solution plus sécurisée consisterait à utiliser des cookies sécurisés avec les attributs `HttpOnly` et `Secure`, afin d'empêcher l'accès au token depuis JavaScript et de limiter les risques d'exploitation.

E. VERSIONNING ET GESTION DE CODE (GIT / GITHUB)

Le projet a été versionné tout au long du développement à l'aide de **Git** pour la gestion locale du code et de **GitHub** comme dépôt distant.

Cette approche a permis de garantir la traçabilité, la sécurité

Organisation du dépôt

- **Dépôt distant** : GitHub
- **Nom du repository** : sav-dev
- **Branches principales** :
 - `main` → version stable
 - `dev` → branche de développement et d'intégration des nouvelles fonctionnalités

5. CAHIER DES CHARGES

A. BESOINS IDENTIFIES

Le processus initial reposait sur Excel :

- Saisie manuelle d'informations client et produit.
- Impression et archivage papier.

Problèmes constatés :

- Doublons et erreurs fréquentes.
- Temps de recherche long.
- Aucune traçabilité.
- Données non sécurisées.

Solution attendue :

- Application web centralisée et sécurisée.
- Recherche intelligente.
- Signature électronique et PDF automatique.
- Évolutivité vers Sage X3.

B. OBJECTIFS

- Digitaliser la création et le suivi des fiches SAV.
- Améliorer la qualité et la sécurité des données.
- Authentification des utilisateurs.
- Réduction de la dépendance à Excel.

C. CIBLES ET UTILISATEURS

1. **Agents SAV** → Création et suivi des fiches.
2. **Responsable SAV** → Supervision et reporting.
3. **Clients** → Signature et réception de fiche PDF/
4. **Fournisseurs** → alertes mails

D. USERS STORIES

Après les interviews utilisateurs, ainsi que suite à plusieurs e-mails reçus (dont voici un exemple) :

Circuit simplifié d'un produit en SAV

Bonjour Sammy,

Ça va ?

Une petite explication pour le circuit sav que j'ai fait de façon simplifiée (je pense...) :

1. **Dépôt du produit par le client au SAV (fiche déjà enregistrée, signée, validée avec le statut ticket sav avec produit)**
 - Point de départ du circuit.
 - Le produit est enregistré dans le système et un mail de demande récupération est envoyé au fournisseur
 - Le mail envoyé, le statut passe : **"En attente de récupération"**. (maj jira)
2. **Délai maximum pour récupération fournisseur : 7 jours**
 - Si le fournisseur ne récupère pas le produit dans ce délai, et que le statut est toujours "en attente de récupération" :
 - ✓ **Relance 1** : Envoi d'un mail de relance au fournisseur mentionnant que 7 jours sont déjà écoulés.
3. **Produit récupéré par le fournisseur : statut « produit chez le fournisseur » (maj jira)**
 - Deux cas :
 - ✓ **Avant 7 jours** → Relance au bout de 7 jours pour demander des nouvelles sur l'état de réparation.
 - ⚠ **Après 7 jours** → Relance après 4 jours (car prise en charge en retard).
4. **Relance continue durant la réparation**
 - ✓ **Relance 2** : 10 jours après la première relance pour informer que le délai de traitement approche de la limite.
 - Délai maximal :
 - 30 jours pour un traitement standard.
 - 45 jours si pièces en commande (à condition que l'ESAV ait informé qu'une pièce est commandée).
5. **Traitement terminé - retour produit**
 - Dès réception du produit statut :
 - ✓ **Maj Jira** « produit mis à disposition ». Envoi d'un mail (automatique sur Jira) au client pour l'informer que le produit est disponible
6. **Cas particuliers à gérer**
 - **Commande de pièce** :
 - Statut « pièce en commande » dans le formulaire → Le client est informé par mail. Pas de changement statut jira
 - Les mêmes relances sont maintenues pour rester dans les délais.
7. **Sorties possibles du cycle SAV**

J'ai utilisé un KANBAN JIRA pour créer les tâches et les US

Mes US et mes tâches :

Projets / Sammy Kanban / Ajouter epic / SGJ-1

Interview Responsable SAV

+

Description

Modifier la description

Activité

Tout

Commentaires

Historique

Journal du travail

SG

Ajouter un commentaire...

Suggérer une réponse...

Puis-je obtenir plus d'informations ?

Mise à jour d'état...

Conseil de pro : appuyez sur **M** pour commenter

SG

GOULJIAR Sammy

2 octobre 2025 à 10:03

Interview en one to one pour comprendre le besoin, dresser les 1er contour du projet

...

Terminé(e) ✓ Terminé ⚡ Améliorer le ticket

Détails

Personne assignée

SG GOULJIAR Sammy

Étiquettes

Aucun

Parent

Aucun

Date d'échéance

18 août 2025

Team

Aucun

Start date

18 août 2025

Développement

Ouvrir avec VS Code

Créer une branche

Créer un commit

Livraisons

Ajouter un feature flag

Rapporteur

SG GOULJIAR Sammy

Sammy Kanban

Projets / Sammy Kanban / Ajouter epic / SGJ-2

Analyse du fichier Excel de gestion du SAV

+

Description

prendre connaissance du fichier,

voir les différentes fonctionnalité existante

Activité

Tout

Commentaires

Historique

Journal du travail

SG

Ajouter un commentaire...

Mise à jour d'état...

Merci...

D'accord...

Conseil de pro : appuyez sur **M** pour commenter

Terminé(e) ✓ Terminé ⚡ Améliorer le ticket

Détails

Personne assignée

SG GOULJIAR Sammy

Étiquettes

Aucun

Parent

Aucun

Date d'échéance

19 août 2025

Team

Aucun

Start date

19 août 2025

Développement

Ouvrir avec VS Code

Créer une branche

Créer un commit

Livraisons

Ajouter un feature flag

Rapporteur

SG GOULJIAR Sammy

Projets / Sammy Karban / Ajouter epic / SGJ-3

Récupération des tickets Jira (projet SER - composant SAV)

+ -

Description

Description :

En tant que Responsable SAV

Je veux avoir la possibilité de récupérer les tickets Jira selon des critères précis

Afin de identifier rapidement les demandes SAV encore en cours et éviter les ressaisies manuelles.

- Seuls les tickets du projet SER (Service Client Ravate.com | Electroménager, High Tech, Mobilier, Déco, Bricolage) sont récupérés.
- Seuls les tickets ayant le composant "SAV" sont récupérés.
- seul les tickets : Ticket SAV avec produit sont affichés et peuvent être récupérés.
- Les tickets récupérés affichent a minima : numéro Jira, résumé, statut, client associé.
- L'actualisation doit permettre de récupérer les nouveaux tickets sans doublons.

Tickets enfant

... ⚙ +

Progression : 0 %

Tickets	Prio...	Pers...	État
SGJ-4 Connexion API JIRA	B...	GO	À FAIRE
SGJ-5 Filtrage des tickets (projet SER, composant SAV, statuts actifs)	B...	Nr	À FAIRE
SGJ-6 Affichage des tickets dans l'application SAV	B...	Nr	À FAIRE
SGJ-7 Création de fiche SAV depuis un ticket Jira	B...	Nr	À FAIRE
SGJ-8 Pré-remplissage automatique des champs	B...	Nr	À FAIRE
SGJ-9 Vérification de l'existence du client (email)	B...	Nr	À FAIRE
SGJ-10 Modification manuelle avant validation	B...	Nr	À FAIRE
SGJ-11 Tests et validation fonctionnelle	B...	Nr	À FAIRE

Terminé(e) Terminé Améliorer le ticket

Détails

Personne assignée	GO GOULJIAR Sammy
Étiquettes	Aucun
Parent	Aucun
Date d'échéance	26 août 2025
Team	Aucun
Start date	25 août 2025
Développement	Ouvrir avec VS Code Créer une branche Créer un commit
Livraisons	Ajouter un feature flag
Rapporteur	GO GOULJIAR Sammy
BigPicture - Skills	Ouvrir BigPicture - Skills
CucumberStudio scenarios	Ouvrir CucumberStudio scenarios
Quantify	Ouvrir Quantify

Automatisation

Exécutions de règles récentes

Projets / Sammy Karban / Ajouter epic / SGJ-12

Gestion des fiches SAV (Champs obligatoires)

+ -

Description

Description :

En tant que Responsable SAV ou Assistante SAV,

Je veux que les champs Désignation, EAN et Fournisseur soient automatiquement remplis en fonction du code RAV (code article),

Et que, s'il existe plusieurs fournisseurs pour le même article, le fournisseur principal soit sélectionné par défaut avec la possibilité de le modifier manuellement,

et que : les champs Numéro de Série, Date d'achat, Panne soient obligatoire

et que : J'aie la possibilité d'ajouter jusqu'à 3 photos du produit, afin de conserver une preuve visuelle du SAV (stockées en base ou dans un dossier lié).

Afin de éviter les saisies manuelles inutiles et de réduire les erreurs de frappe et garantir une meilleure traçabilité des interventions.

Tickets enfant

... ⚙ +

Progression : 100 %

Tickets	Priorité	Personne assignée	État
SGJ-13 Création de la requête SQL et accès aux données articles	Bas	GO GOULJIAR Sa...	TERMINÉ(E)
SGJ-14 Développement backend (API Base Article)	Bas	GO GOULJIAR Sa...	TERMINÉ(E)
SGJ-15 Intégration frontend (formulaire de fiche SAV)	Bas	GO GOULJIAR Sa...	TERMINÉ(E)
SGJ-16 Tests unitaires et validation	Bas	GO GOULJIAR Sa...	TERMINÉ(E)

Terminé(e) Terminé Améliorer le ticket

Détails

Personne assignée	GO GOULJIAR Sammy
Étiquettes	Aucun
Parent	Aucun
Date d'échéance	10 sept. 2025
Team	Aucun
Start date	10 sept. 2025
Développement	Ouvrir avec VS Code Créer une branche Créer un commit
Livraisons	Ajouter un feature flag
Rapporteur	GO GOULJIAR Sammy
BigPicture - Skills	Ouvrir BigPicture - Skills
CucumberStudio scenarios	Ouvrir CucumberStudio scenarios
Quantify	Ouvrir Quantify

Automatisation

Projets /
 Sammy Kanban /
 Ajouter epic /
 SGJ-17

Gestion du workflow à la création de fiche

Description

Description :

En tant que Responsable SAV ou Assistante SAV,

Je veux que, lors du clic sur le bouton "Création fiche",

Les champs préalablement remplis soient enregistrés dans la base de données et qu'une date de création soit automatiquement ajoutée,

Afin de sauvegarder les fiches SAV et garantir leur traçabilité.

Critères d'acceptation

- Tous les champs saisis dans le formulaire (dont les champs obligatoires) sont bien enregistrés.
- La date de création est automatiquement générée côté backend (et non saisie manuellement).
- La fiche SAV est enregistrée en base avec un **identifiant unique**.
- Un message de confirmation (ou notification) indique à l'utilisateur que la fiche a bien été créée.
- En cas d'erreur (ex. problème de base de données), un message d'erreur clair est affiché.

Activité

Tout
Commentaires
Historique
Journal du travail

Ajouter un commentaire...

Mise à jour d'état...
Merci...
D'accord...

Conseil de pro : appuyez sur pour commenter

À faire
⚡
Améliorer le tic

Détails

Personne assignée
Étiquettes
Parent
Date d'échéance
Team
Start date
Développement
Livraisons
Rapporteur
BigPicture - Skills
CucumberStudio scenarios
Quantify

Automatisation

Exécutions de règles récentes

Gestion du workflow de validation et synchronisation avec Jira

Description

Description :

En tant que **Responsable SAV**,

Je veux que le ticket "Créer" reprenne le **statut actuel du ticket dans Jira**,

Et qu'un **filtre** permette de **changer ce statut**,

Avec un **bouton "Valider"** pour enregistrer la modification.

Lors de la validation :

- L'application doit **mettre à jour automatiquement le statut de l'issue dans Jira**.
- À la **signature du client et du SAV**, après avoir cliqué sur "Valider la signature" :
 - Un **workflow** se déclenche,
 - La **fiche est bloquée** (via la mise à jour d'un champ en base de données),
 - Le statut du ticket passe à **"En attente récupération fournisseur"**,
 - Un **mail est envoyé au fournisseur** pour l'informer que le produit est disponible à l'entrepôt.

Une **fiche validée n'est plus modifiable**.

Afin de :

- Garantir la cohérence** entre les statuts Jira et les fiches SAV.
- Sécuriser le processus** après validation (aucune modification possible).
- Notifier automatiquement** le fournisseur dès qu'un produit est prêt à être récupéré.

Activité

Projets /  Sammy Kanban /  Ajouter epic /  SGJ-19

Recherche et consultation des fiches SAV



Description

Description :

En tant que **Responsable SAV ou Assistant SAV**,

Je veux pouvoir **rechercher une fiche dans l'application** à partir des champs suivants :

- **ID Fiche** (identifiant unique créé automatiquement),
- **Nom**,
- **Prénom**,
- **Téléphone (1 ou 2)**,
- **Numéro de ticket SER (Jira)**,

Afin de **retrouver rapidement une fiche existante parmi l'ensemble des fiches créées**.

Au clic sur le bouton **"Rechercher"** :

- Le **filtre** s'applique pour afficher uniquement les fiches correspondant aux critères saisis.
- La page doit lister **toutes les fiches SAV existantes**, avec possibilité de filtrer dynamiquement.
- À côté de chaque fiche, un bouton **"Voir la fiche"** doit apparaître.
- En cliquant sur ce bouton, l'utilisateur est **redirigé vers la fiche SAV correspondante** (vue détaillée).

Afin de :

- **Faciliter la recherche et le suivi** des fiches SAV existantes.
- **Gagner du temps** dans la gestion quotidienne du service.
- **Améliorer la traçabilité** et l'accès rapide aux informations clients et tickets Jira associés.



Ajouter un commentaire...

Qui travaille sur ce ticket ? Mise à jour d'état Merci

A f

Dét

Pei

Éti

Pa

Da

Tei

Sta

Dé

Liv

Raj

Big

Cu

Qu

Aut

Projets /  Sammy Kanban /  Ajouter epic /  SGJ-20

Suivi et relance automatique des fournisseurs (workflow SAV)



Description

Description :

En tant que **Responsable SAV**,

Je veux pouvoir **visualiser et suivre les tickets SAV** selon leur **statut et leur ancienneté**,

Afin de **relancer automatiquement les fournisseurs et garantir le respect des délais de traitement**.

Règles de gestion / Workflow attendu :

1. Statut : "En attente de récupération fournisseur"

- Afficher les tickets dont le statut est "En attente de récupération fournisseur" **depuis plus de 7 jours**.
- Permettre l'envoi automatique ou manuel d'un **email de relance au fournisseur** (via un template).

2. Statut : "Produit chez le fournisseur"

- **Avant 7 jours** : première relance automatique au bout de **7 jours** pour demander l'état d'avancement de la réparation.
- **Après 7 jours** : relance suivante au bout de **4 jours supplémentaires** (prise en charge en retard).
- **Relance continue** :
 - Une deuxième relance est envoyée **10 jours après la première** pour rappeler la limite du délai de traitement.

3. Délais maximum de traitement

- **30 jours** : délai standard pour un traitement complet.
- **45 jours** : délai prolongé si une **pièce est en commande** (à condition que le fournisseur ait signalé la commande).

4. Fin du traitement / retour produit

- Dès réception du produit réparé :
 - Mise à jour du **statut Jira** → "**Produit mis à disposition**".
 - Envoi automatique d'un **email au client** pour l'informer que le produit est disponible à l'entrepôt.

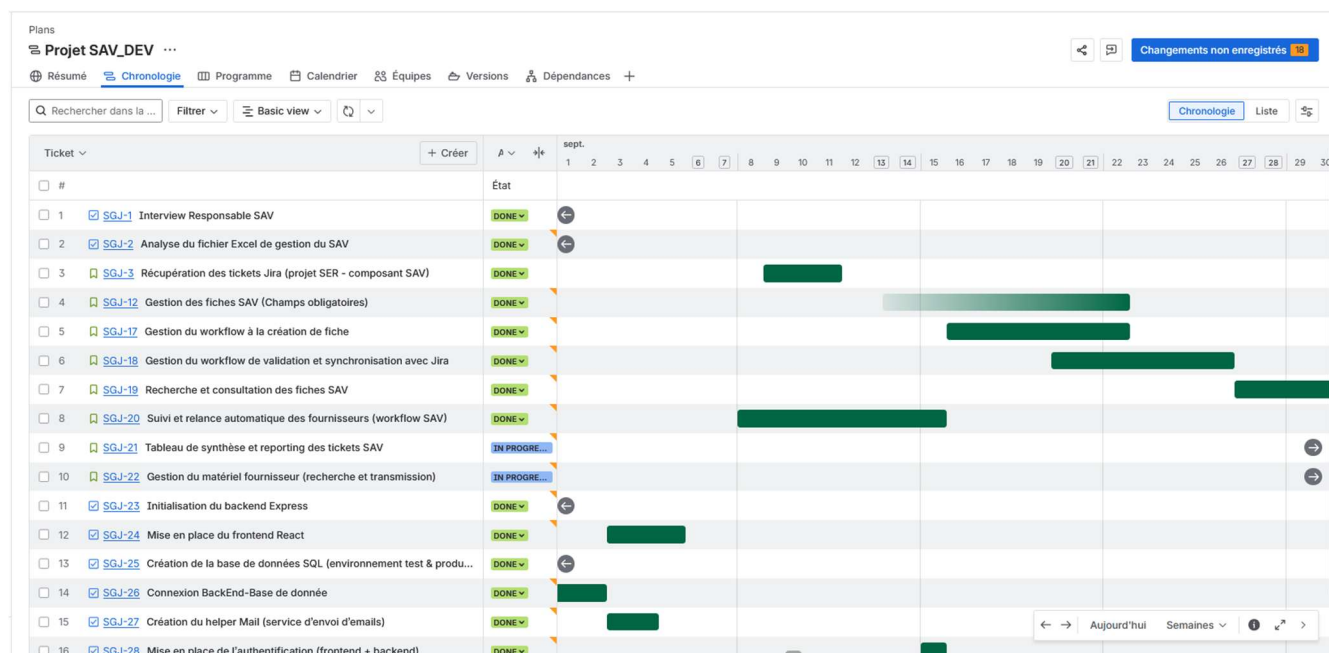
5. Cas particuliers – Commande de pièce

- Si le statut est "Pièce en commande" :
 - Envoi d'un **mail informatif au client** (pièce en attente).
 - **Aucun changement de statut Jira**.
 - Les relances fournisseurs restent actives pour maintenir le suivi.

Afin de :

- **Assurer un suivi rigoureux des délais de traitement SAV**

DIAGRAMME DE GANTT



E. MVP (PRODUIT MINIMUM VIABLE)

- Authentification sécurisée (JWT + bcryptjs).
- CRUD complet des fiches SAV.
- Détection des doublons client.
- Génération PDF.
- Signature électronique.

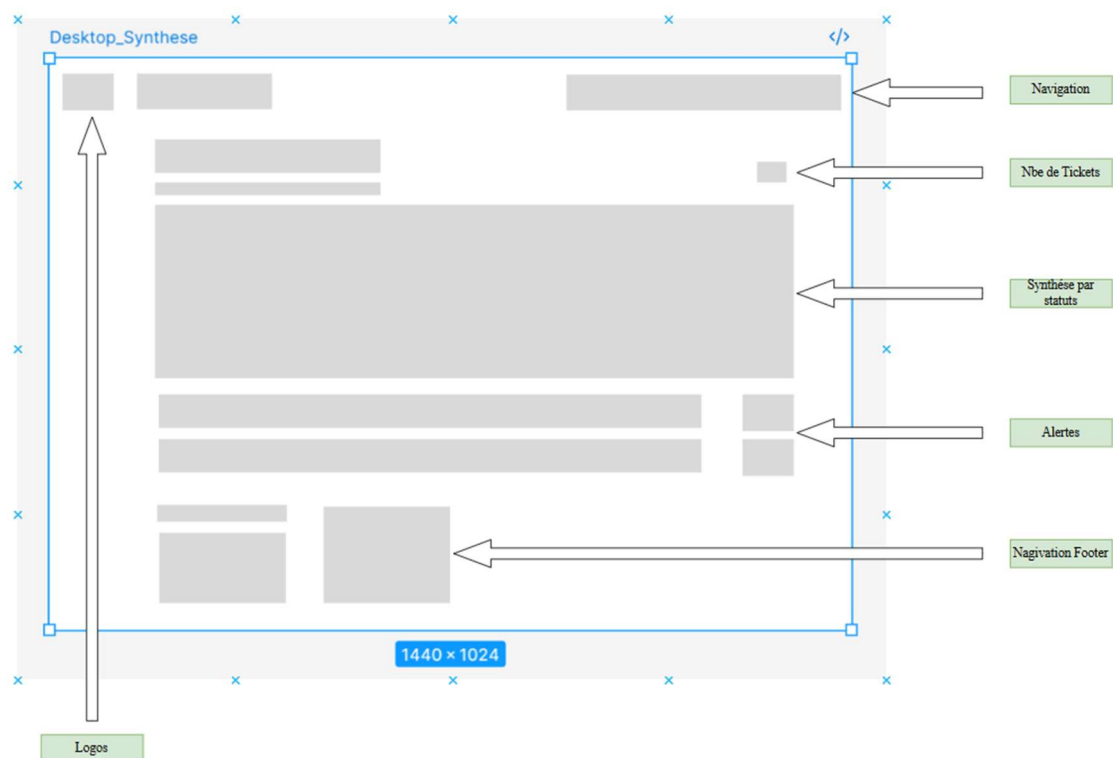
F. ÉVOLUTIONS POTENTIELLES

- Synchronisation avec Sage X3.
- Tableau de bord Power BI.
- Envoi automatique de fiches par email. (surveillance de la base de donnée, power automate CRON?)

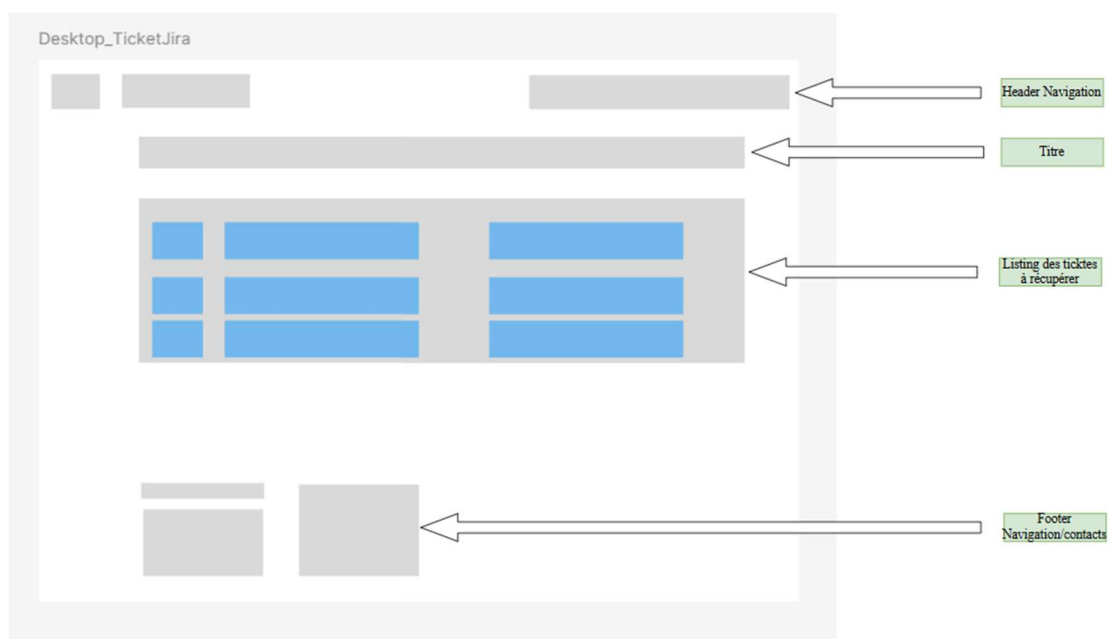
6. WIREFRAMES ET MAQUETTES

Desktop

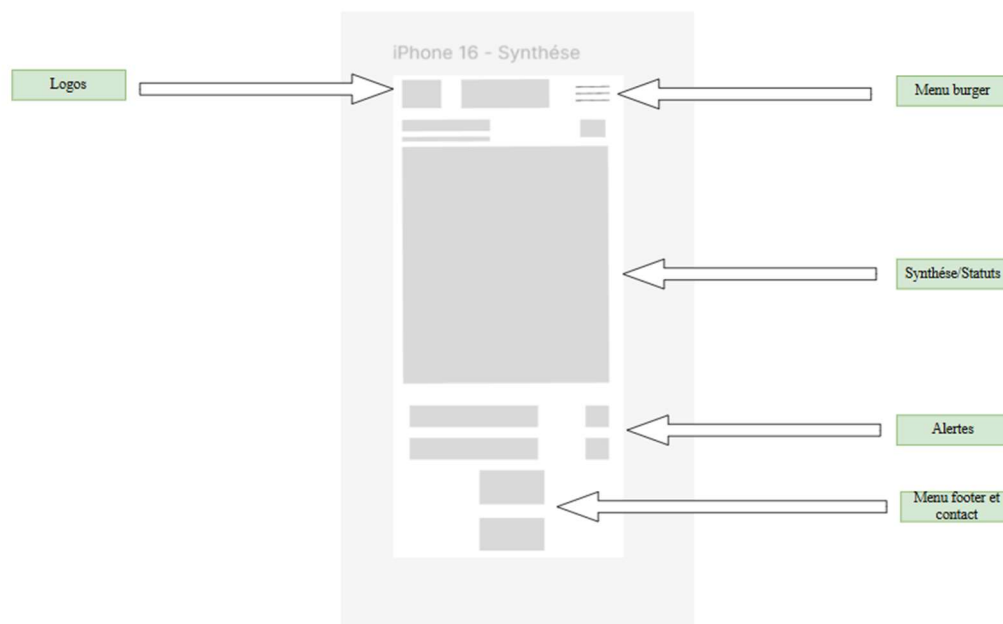
Page de redirection après connexion :



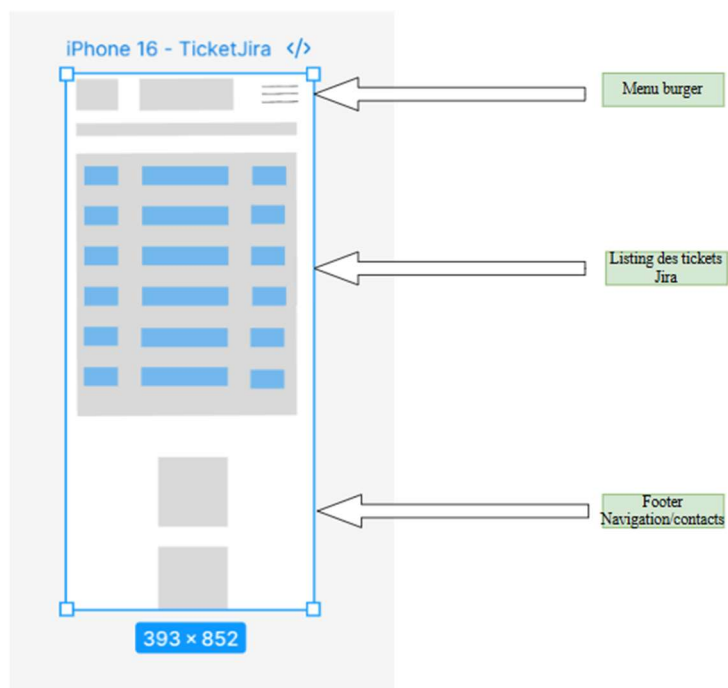
Page Ticket JIRA

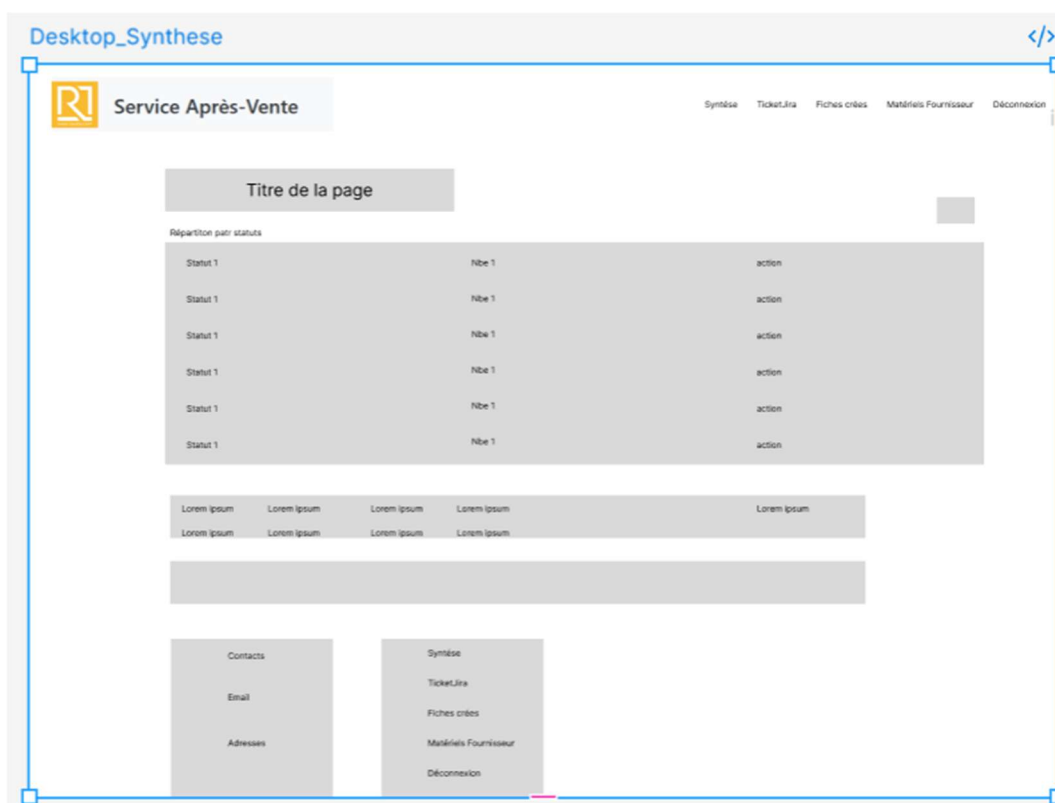


Smartphone



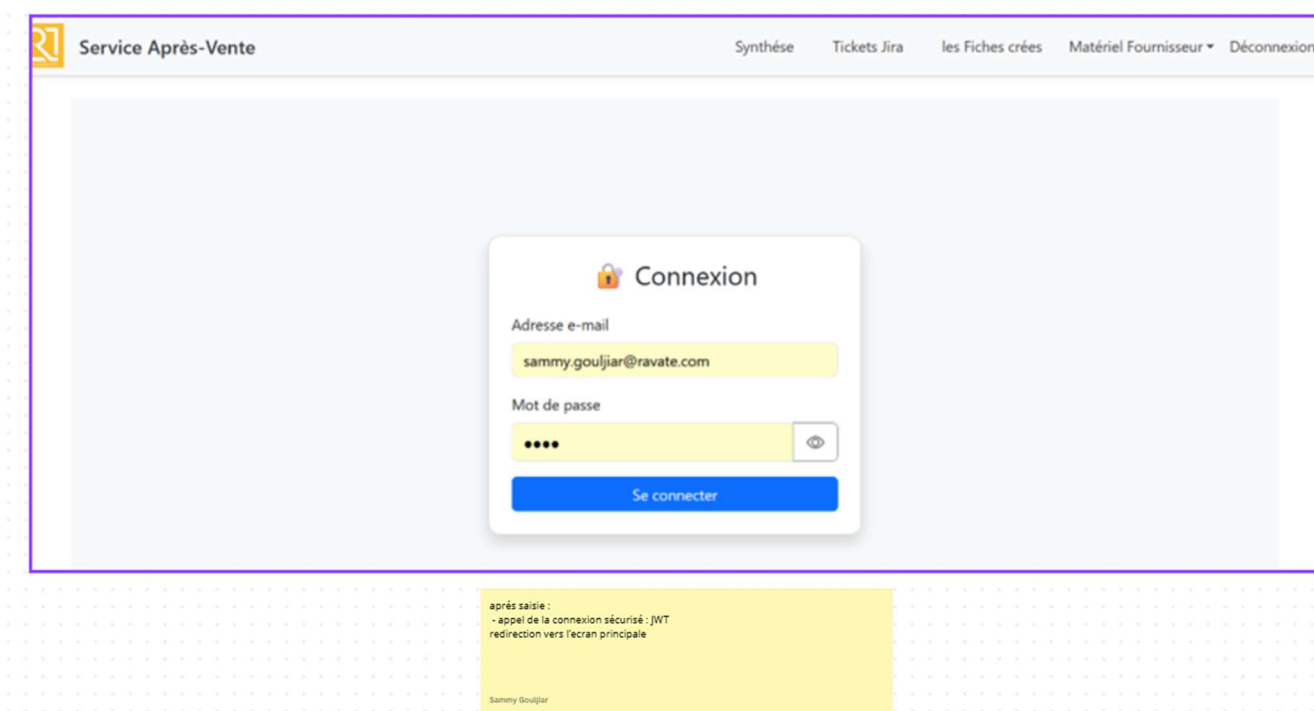
Page Tickets JIRA





Les écrans :

1 - Connexion



2 – Page de synthèse

Bienvenue Sammy !
Tableau de bord - Synthèse
Vue d'ensemble des fiches par statut.

Répartition par statut

Statut	Nombre	Action
PRODUIT CHEZ LE FOURNISSEUR	6	Voir les fiches
ATTENTE RÉCUP FOURNISSEUR	4	Voir les fiches
PRODUIT MISE À DISPOSITION	1	Voir les fiches
RÉCUPÉRÉ	1	Voir les fiches
CLÔTURÉ REMBOURSEMENT	1	Voir les fiches

⚠️ Délai max récupération fournisseur dépassé (7 jours)

ID Fiche	Numéro Série	Fournisseur	État	Délai (jours)	Action
115	SER-80653	REUNION TECHNOLOGIE DISTRIBUTION	attente récup fournisseur	8	Envoyer mail

Contact
Téléphone : 0692 00 00 00
Email : sav.distribution@ravata.com
Service Après-Vente

Synthèse
Tickets Jira
Les Fiches créés
Matériel Fournisseur

Fetch de l'API vers la base de donnée pour récupérer la totalité des tickets (methode GET)
Fetch de l'API JIRA pour récupérer le statut des tickets (dans JIRA)
affichage des fiches en anomalies (dates dépassées)

Sammy Gouljlar

3 – Récupération des tickets JIRA

Tickets Jira

Rechercher le numéro du ticket (ex: 79030)

Ticket	Résumé	Statut	Créé le	Client	Action
SER-80173	Service apres-vente - 119605 - Laurenc...	Ticket SAV sans produit	05/10/2025	Laurence Angila angila.laurence@reynolds.net	Créer fiche
SER-80448	Service apres-vente - 154403 - Jean ho...	Ticket SAV sans produit	29/09/2025	Jean Hoarau jean.hoarau@reynolds.com	Créer fiche
SER-79030	Nouvelle commande collaborateur - M...	Ticket SAV sans produit	26/08/2025	Sammy Gouljlar benoit.moussard@gmail.com	Créer fiche

Contact
Téléphone : 0692 00 00 00
Email : sav.distribution@ravata.com
Service Après-Vente

Synthèse
Tickets Jira
Les Fiches créés
Matériel Fournisseur
Déconnexion

© 2025 DigDae - Tous droits réservés.

Fetch vers l'API JIRA pour récupérer les tickets filtrer selon le KANBAN et le statut défini dans le back

Sammy Gouljlar

4- Création d'une fiche SAV

Recherche de fiches SAV

Rechercher

Créer une nouvelle fiche SAV

Informations client

Nom

Prénom

Téléphone 1

Téléphone 2

Adresse

Code Postal

Ville

Email

Photos (max 3, facultatif)

Détails de la fiche

Code RAV

Désignation

EAN

Fournisseur

Numéro de série

Date d'achat

Numéro Commande

Numéro SER (issu de Jira)

Panne

Service Après Vente

Recherche de fiches SAV

Rechercher

Liste des Fiches

120

VINCESAUP

Overseas

305-81212

Rechercher cette fiche

Activer

121

MICHELLE RONALD

Maria

305-80889

Rechercher cette fiche

Activer

122

Patrice

Karin

305-81212

Rechercher cette fiche

Activer

123

Jean

Catherine

305-80889

Rechercher cette fiche

Activer

124

BRIGIDE

KELLY

305-80889

Rechercher cette fiche

Activer

125

Martine

Nadia

305-80889

Rechercher cette fiche

Activer

126

CHLOE

JEROME

305-80814

Rechercher cette fiche

Activer

127

Marc

Typhaine

305-80544

Rechercher cette fiche

Activer

128

SAUTRON SASU CASE MARIANNE CROCH

Audrey

305-76087

Rechercher cette fiche

Activer

129

Angela

Frédérique

305-76024

Rechercher cette fiche

Activer

130

Sabine

SARAH

305-76088

Rechercher cette fiche

Activer

131

Chang

Mari

305-76087

Rechercher cette fiche

Activer

132

Rangana

Typhaine

305-76216

Rechercher cette fiche

Activer

Contact

Téléphone : 06 00 00 00 00

Email : contact@service.com

Service Après Vente

Service

Tickets Jira

Les Fiches index

Statistiques

Connexion

Digitalisation du SAV – FRED SAMMY GOULJIAR – 07/26

35

5 – Fournisseurs

Service Après-Vente

Synthèse

Tickets Jira

les Fiches créés

Matériel Fournisseur ▾

Déconnexion

Listing Fournisseurs

Tous les fournisseurs ▾

Valider	ID	Fiche	Ticket	Numéro de commande	Code fournisseur	EAN	Désignation	Fournisseur	Fiche transmission
☒	115	SER-80653	129482	1271949	4895222704500	MONTMOSWZ70-PB 1271949	REUNION TECHNOLOGIE DISTRIBUTION	—	
☒	121	SER-80668	159729	1271949	4895222700359	MONTMOSWZ70-RG 1271949	REUNION TECHNOLOGIE DISTRIBUTION	—	
☒	123	SER-81272	141456	1270204	4718017840989	PORT ASUS 17 VIVOBOK M710D	RANY SAS	—	
☒	124	SER-80893	—	3003636	8806098659463	OLED 165C UHD 4K HDR STV A	COMEBACK SAS	—	

7. SPÉCIFICATIONS TECHNIQUES

A. TECHNOLOGIES UTILISEES

- **Front-end : React, HTML, CSS, Bootstrap**
- **Back-end : Node.js, Express**
- **Base de données : MySQL**
- **Sécurité : JWT, bcryptjs**
- **PDF : html2pdf.js**
- **Fichiers : stockage images (signatures)**
- **Mail : mailjet**
- **API : node-fetch**
- **gestion de la configuration via .env**
- **Postman pour tester les routes**

B. NAVIGATEURS COMPATIBLES

Chrome • Firefox • Edge • Safari

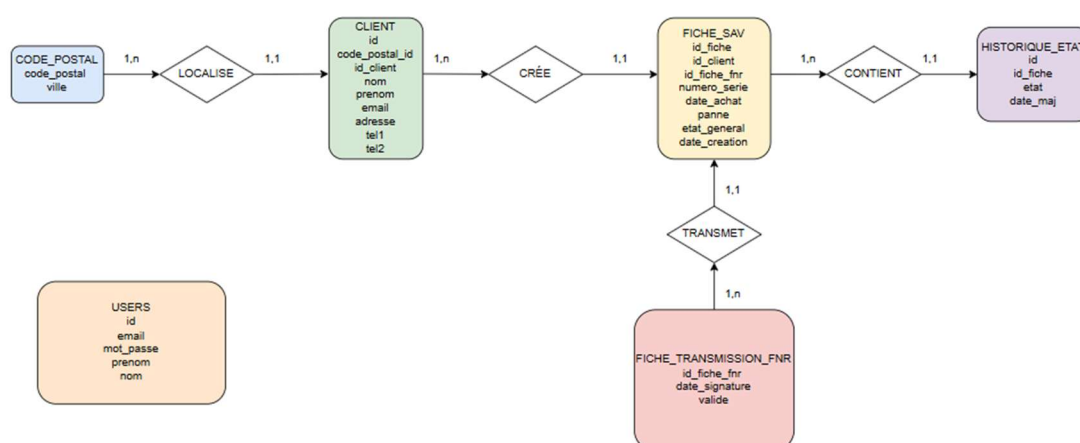
C. DEPLOIEMENT ENVISAGE

1. Interne réseau local
2. Interfaçage Sage X3 (long terme)

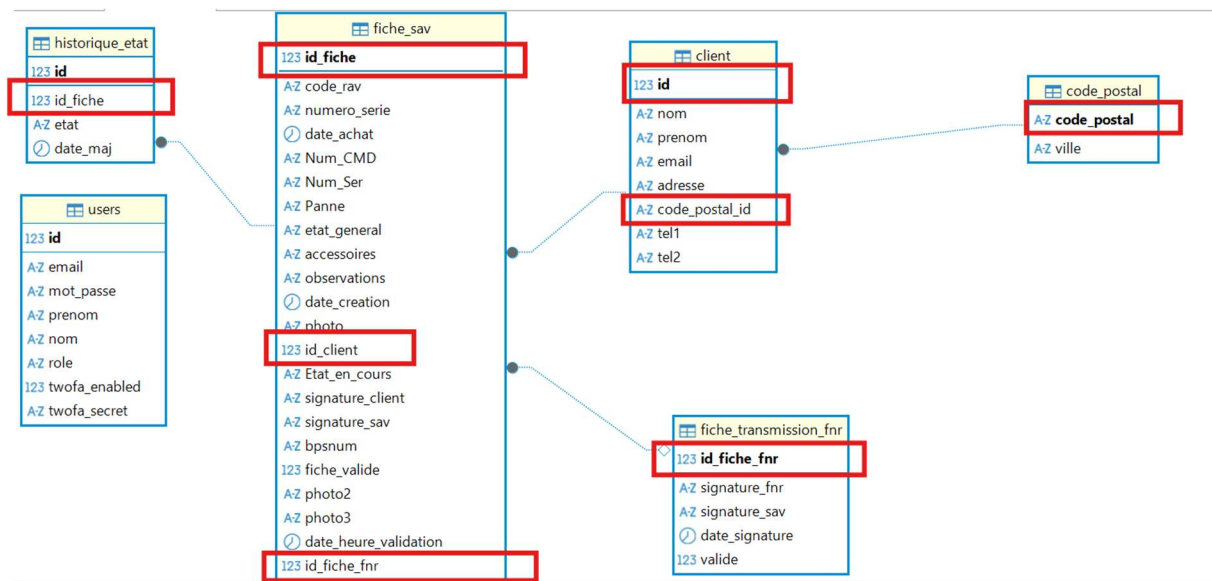
D. BASE DE DONNEES

- Table fiche_sav : gestion des fiches (CRUD, photos (via lien url), interventions...)
- Table fiche_transmission_fnr : gestion des transmissions aux fournisseurs
- Table code_postal : normalisation.
- Table historique_etat : gérer les historiques sur les différents états
- Table client : gérer les clients (éviter les doublons via email)
- Users : gérer les utilisateurs (nom prenom email à terme rôles...)

Le MCD :



LE MLD



Descriptif

CODE_POSTAL

(code_postal, ville)

- code_postal : VARCHAR(10), **PK**
- ville : VARCHAR(100)

Relations :

- 1 CODE_POSTAL peut être lié à plusieurs CLIENT.

CLIENT

(id, nom, prenom, email, adresse, code_postal_id, tel1, tel2)

- id : INT, **PK**
- nom, prenom, email : NOT NULL
- code_postal_id : VARCHAR(10), **FK** → **CODE_POSTAL**(code_postal)
- email : UNIQUE

Relations :

- Plusieurs CLIENT partagent un même CODE_POSTAL.
- 1 CLIENT peut avoir plusieurs FICHE_SAV.

FICHE_TRANSMISSION_FNR**(id_fiche_fnr, signature_fnr, signature_sav, date_signature, valide)**

- id_fiche_fnr : INT, **PK**
- valide : TINYINT(1) (par défaut 1)

Relations :

- 1 FICHE_TRANSMISSION_FNR peut être reliée à plusieurs FICHE_SAV

FICHE_SAV**(id_fiche, code_rav, numero_serie, date_achat, Num_CMD, Num_Ser, Panne, etat_general, accessoires, observations, date_creation, photo, id_client, Etat_en_cours, signature_client, signature_sav, bpsnum, fiche_valide, photo2, photo3, date_heure_validation, id_fiche_fnr, date_passage_etat_en_cours, fournisseur, date_recup, dateChezFnr, date_mail_attente, date_mail_recup)**

- id_fiche : INT, **PK**
- id_client : INT, **FK** → **CLIENT(id)**
- id_fiche_fnr : INT, **FK** → **FICHE_TRANSMISSION_FNR(id_fiche_fnr)**
- Num_Ser : UNIQUE
- date_creation : DATETIME, valeur par défaut CURRENT_TIMESTAMP

Relations :

- 1 CLIENT → n FICHE_SAV
- 1 FICHE_TRANSMISSION_FNR → n FICHE_SAV
- 1 FICHE_SAV → n HISTORIQUE_ETAT

HISTORIQUE_ETAT**(id, id_fiche, etat, date_maj)**

- id : INT, **PK**
- id_fiche : INT, **FK** → **FICHE_SAV(id_fiche)**
- etat : VARCHAR(100)
- date_maj : DATETIME, par défaut CURRENT_TIMESTAMP

Relations :

- Plusieurs HISTORIQUE_ETAT peuvent appartenir à une même FICHE_SAV.

USERS

(id, email, mot_passe, prenom, nom)

- id : INT, **PK**
- email : UNIQUE
- mot_passe : VARCHAR(200)

Relations :

- Entité indépendante (pas de FK). Sert pour l'authentification.

Résumé des cardinalités :

Table source	Table cible	Type de relation	Cardinalité	Clé étrangère
code_postal	client	1 → n	Un code postal peut être associé à plusieurs clients	client.code_postal_id → code_postal.code_postal
client	fiche_sav	1 → n	Un client peut avoir plusieurs fiches SAV	fiche_sav.id_client → client.id
fiche_transmission_fnr	fiche_sav	1 → n	Un envoi fournisseur peut concerner plusieurs fiches SAV	fiche_sav.id_fiche_fnr → fiche_transmission_fnr.id_fiche_fnr
fiche_sav	historique_etat	1 → n	Une fiche SAV peut avoir plusieurs états dans le temps	historique_etat.id_fiche → fiche_sav.id_fiche
users	—	—	Table indépendante	—

E. ROUTES

Méthode	Route	Contrôleur / Middleware	Description
POST	/insertlogin	UserController.adduser	Crée un nouvel utilisateur (inscription). Le mot de passe est haché avant insertion en base.
POST	/login	UserController.login	Authentifie un utilisateur avec son email et son mot de passe. Retourne un token JWT en cas de succès.
GET	/profile	verifyToken, puis callback (req, res)	Exemple de route protégée : renvoie les infos du profil associées au token JWT fourni.
GET	/liste	FicheController.getAllFiches	Récupère la liste complète des fiches SAV.
GET	/recherche	FicheController.rechercheFiche	Recherche de fiches selon des critères dynamiques (numéro série, client, etc.).
GET	/delai	FicheController.getCalculDelai	Calcule le délai global de traitement d'une fiche.
GET	/delai-produit-chez-fnr	FicheController.getCalculDelaiPdtChezFnr	Calcule le délai de présence du produit chez le fournisseur.
GET	/delai-historique	FicheController.getCalculDepuisHistorique	Calcule les délais à partir de l'historique des états.
GET	/designations/:coderav	getFilteredData	Récupère les désignations d'article (Sage X3).
GET	/client-par-email	FicheController.getClientByEmail	Récupère un client existant via son email.
GET	/fournisseurs	ListeFournisseursController.getFournisseurs	Récupère la liste des fournisseurs.
GET	/historique/:id	FicheController.getHistoriqueById	Récupère l'historique des changements d'état d'une fiche.
GET	/ville/:code_postal	FicheController.getVilleByCodePostal	Renvoie la ville associée à un code postal.
GET	/regroupement/statut	verifyToken, FicheController.getRegroupementByStatus	Récupère un regroupement de fiches par statut (route protégée).
GET	/:id	FicheController.getFicheById	Récupère une fiche spécifique par son ID.
PUT	/:id/client	FicheController.updateFicheClient	Met à jour les informations client liées à une fiche.
PUT	/:id/etat	FicheController.updateEtatEnCours	Met à jour l'état en cours d'une fiche.
PUT	/:id/panne	FicheController.updatePanne	Met à jour les détails de la panne.
PUT	/:id/signatures	SignaturesController.updateSignatures	Met à jour les signatures (client / SAV).
PUT	/:id/validation	FicheController.markValidatedNow	Marque la fiche comme validée (date de validation).
PUT	/:id_fiche/dateMailAttente	FicheController.updatedateMail	Met à jour la date d'envoi du mail d'attente.
PUT	/:id_fiche/dateMailRecup	FicheController.updatedateMailRecup	Met à jour la date d'envoi du mail de récupération.
PUT	/:id/locked	LockedFicheController.markLocked	Verrouille la fiche (empêche modification).
PUT	/:id/lockedfnr	LockedFicheFnrController.markLockedFnr	Verrouille la fiche du fournisseur.
POST	/:id_fiche/historiqueEtat	FicheController.updateHistoriqueEtat	Ajoute un nouvel état dans l'historique de la fiche (changement de statut).
POST	/fiche_fournisseur	fiche_fournisseur.create	Crée une nouvelle fiche fournisseur (liée à une fiche SAV).
GET	/fiche_fournisseur/transmission	ListeFournisseursController.getFicheFnr	Récupère la liste complète des fiches fournisseur en transmission.
GET	/fiche_fournisseur/transmission/:id	ListeFournisseursController.getFicheFnrById	Récupère une fiche fournisseur spécifique selon son ID.
GET	/fiche_fournisseur/:bpsnum	fiche_fournisseur.getFicheByBpsnum	Récupère une fiche fournisseur selon son numéro BPS.
PUT	/fiche_fournisseur/:id/signature	fichefnr_signature.updateSignatures	Met à jour les signatures fournisseur/SAV sur la fiche fournisseur.
PUT	/fiche_fournisseur/:id_fiche_fnr	fiche_fournisseur.updateFiche	Met à jour les informations générales d'une fiche fournisseur (date, validation, etc.).
GET	/issue/:key	C.getTicket	Récupère les détails complets d'un ticket à partir de sa clé unique (:key).
POST	/issue-ser	verifyToken, C.getLastTickets	Récupère les derniers tickets liés à l'utilisateur connecté (via token JWT).
GET	/issue/:key/transitions	C.listTransitions	Liste toutes les transitions d'état possibles pour un ticket donné (:key).
POST	/issue/:key/status	C.updateStatusByName	Met à jour le statut d'un ticket en précisant le nom du statut cible.
POST	/issue/:key/statusById	C.updateStatusById	Met à jour le statut d'un ticket en précisant directement l'ID du statut cible.
GET	/issue/:issueKey/status	C.getAllStatusController	Récupère tous les statuts disponibles pour un ticket donné (:issueKey).
GET	/field	C.getFields	Récupère la liste des champs disponibles (métadonnées ou propriétés des tickets).

F. API JIRA

Objectif

L'application SAV devait interagir avec le Kanban Jira utilisé par l'équipe de gestion du SAV afin de :

- Récupérer automatiquement les tickets SAV selon un statut et un KANBAN précis,
- Afficher la synthèse des demandes directement dans l'application web,
- Suivre l'évolution des statuts

Cette intégration visait à centraliser les informations SAV et à éviter la double saisie entre Jira et l'application.

Authentification et autorisations

L'accès à l'API Jira nécessite une authentification par jeton personnel (API Token) associé à un compte disposant des droits de lecture sur les projets concernés.

Pour ce faire :

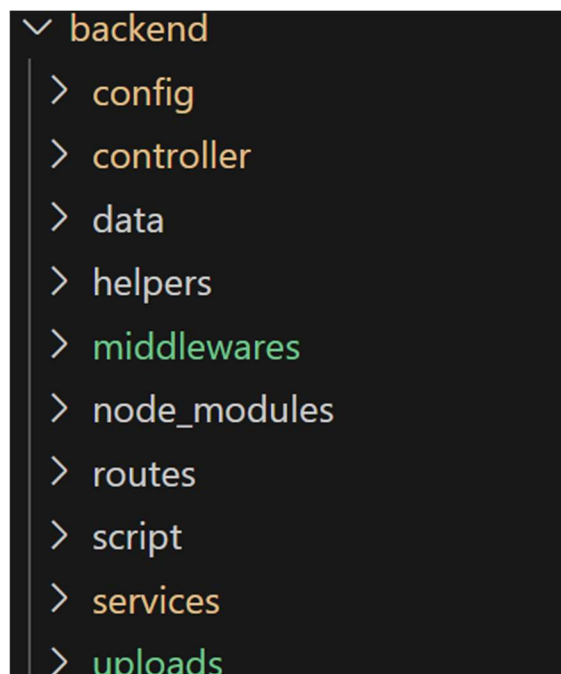
- J'ai créé un jeton d'accès personnel via mon compte Atlassian (valide 1 an),
- J'ai fait valider auprès de l'administrateur Jira l'autorisation d'effectuer des connexions API externes,
- Un Kanban de test dédié m'a été créé afin de réaliser les essais sans impacter les données de production.

8. ARCHITECTURE

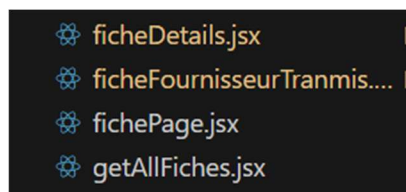
Architecture en MVC (Model View Controller)

L'application SAV Digital a été développée selon le modèle d'architecture MVC, permettant une claire séparation des responsabilités et une meilleure maintenabilité du code.

Backend :



Front end :



Application de la convention camelCase pour le nommage des fichiers jsx

9. SÉCURITÉ ET VEILLE TECHNOLOGIQUE

- Utilisation de **JWT** pour l'authentification.
- Hashage des mots de passe avec **bcryptjs**.
- Mise en place du middleware helmet pour la sécurisation des entêtes http
- Afin de sécuriser l'application et d'éviter l'insertion de données non conformes dans la base de données, la librairie **Joi** a été intégrée au backend pour effectuer une validation des données reçues par l'API.
 - La librairie est installée via la commande suivante :
 - Npm install joi
 - Un middleware de validation a ensuite été mis en place afin de définir un schéma de validation des données. Ce middleware utilise Joi pour vérifier les champs envoyés dans la requête (types, formats, champs obligatoires, longueurs, etc.) avant d'autoriser l'accès au contrôleur au service métier :

```
const Joi = require('joi');

const validationFiche = (req, res, next) => {
  const schema = Joi.object({
    nom: Joi.string().min(2).max(100).required(),
    prenom: Joi.string().min(2).max(100).required(),
    email: Joi.string().email().required()
  });

  const { error } = schema.validate(req.body);

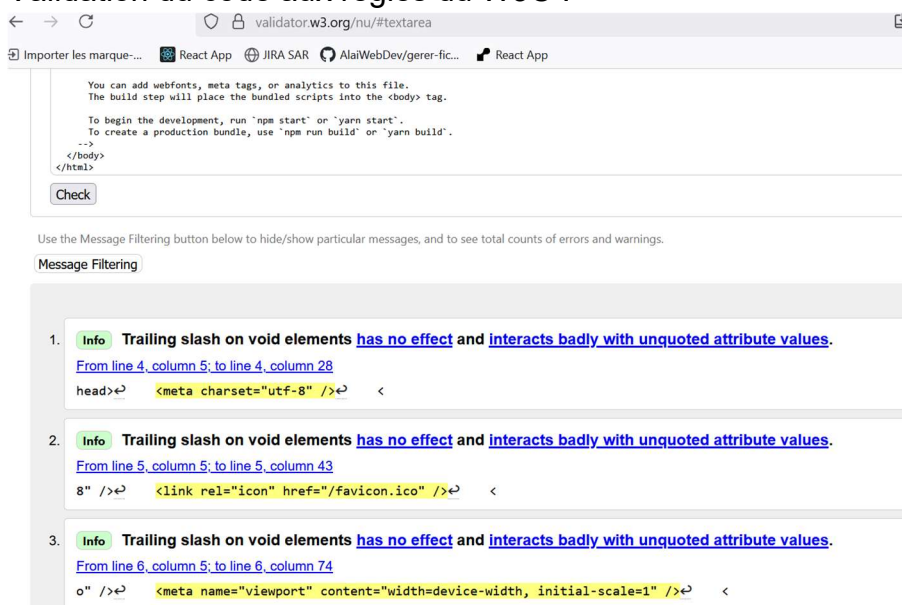
  if (error) {
    return res.status(400).json({
      message: error.details[0].message
    });
  }

  next();
};

module.exports = validationFiche;
```

Nb : Une première validation des champs est réalisée côté frontend afin d'améliorer l'expérience utilisateur et d'éviter l'envoi de données incorrectes. De plus, React protège automatiquement l'affichage dans le DOM en échappant les contenus afin d'éviter certaines injections XSS. Cependant, la validation côté backend reste indispensable, car elle constitue le véritable niveau de sécurité empêchant l'insertion de données invalides ou malveillantes dans la base de données.

- Protection des données utilisateurs.
- Veille sur les technologies : Node LTS, React, Power BI Embedded.
- Validation du code aux règles du W3C :



Perspectives (Version 2)

La version 2 du projet intégrera une couche décisionnelle via Microsoft Power BI, afin de permettre :

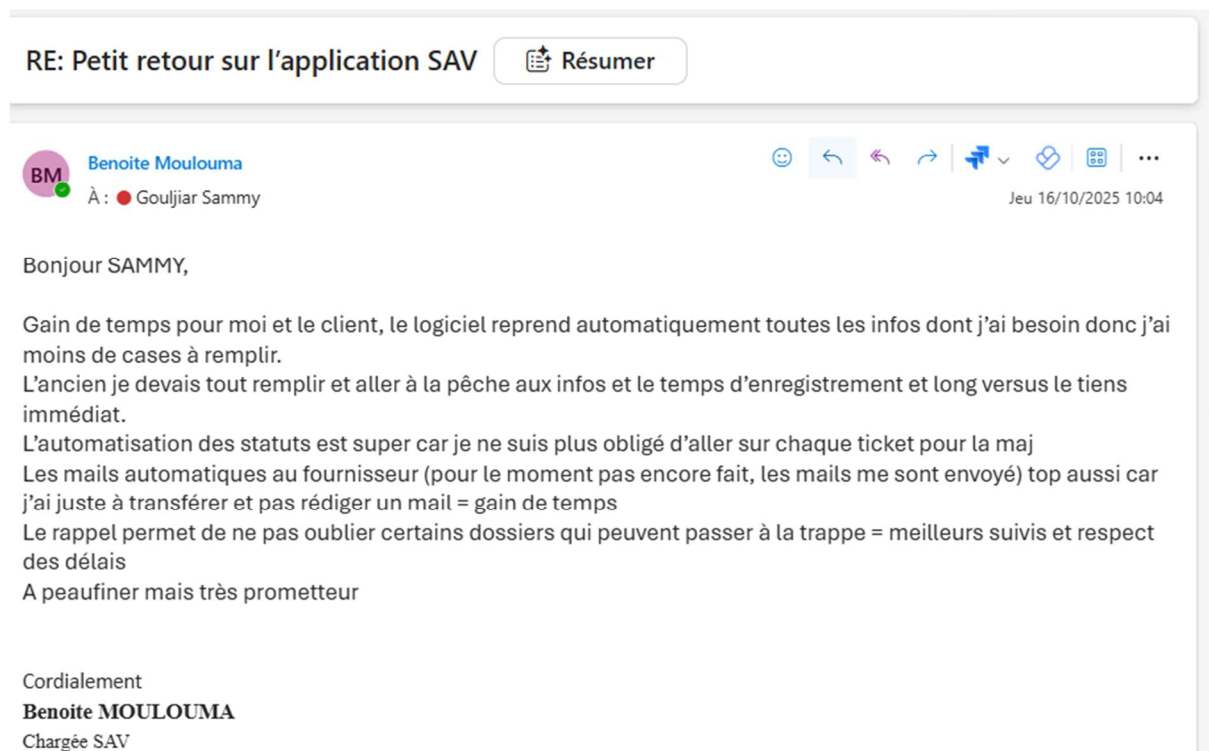
- La visualisation d'indicateurs clés liés à l'activité du SAV (volumes de fiches, délais de traitement, produits récurrents, types de pannes, état général, etc.).
- L'analyse des tendances et récurrences pour améliorer la qualité des produits et le service client.
- Le pilotage en temps réel des performances du SAV grâce à des tableaux de bord dynamiques et personnalisés.

Respect du RGPD :

Le respect du RGPD a été pris en compte dès la conception du projet.

À ce jour, le service RGPD du Groupe Ravate n'a pas encore été contacté, mais le sera avant la mise en production afin de valider la conformité complète du traitement des données personnelles.

Ci-dessous le retour de la responsable en charge du SAV :



10.LIVRABLE

Les livrables du projet ont pour objectif de démontrer la capacité de la solution proposée à répondre à la problématique initiale, à savoir améliorer la gestion des fiches SAV et de faciliter le suivi des interventions.

Le principal livrable est l'application web développée dans le cadre du projet. Celle-ci permet de centraliser les informations liées aux interventions SAV, d'enregistrer les données clients, de suivre les fiches d'intervention et de recueillir les signatures numériques des parties concernées. La génération d'un document PDF permet également de formaliser l'intervention et de conserver une trace exploitable.

Le fichier Excel qui existait auparavant n'apportait pas toute la logique ni la sécurité attendues lors de l'expression du besoin. La mise en œuvre de cette application

confirme la faisabilité technique de la solution et sa compatibilité avec les besoins métier identifiés. Les fonctionnalités développées permettent de structurer les informations, de limiter les erreurs de saisie et d'améliorer la traçabilité des interventions.

11. JEUX D'ESSAI

Mise en place de données de test et validation de l'application

Afin de tester le bon fonctionnement de l'application dans des conditions proches de l'utilisation réelle, un jeu de données fictif a été créé pour simuler l'activité du service après-vente (SAV).

Ce jeu de données comprend :

- 1 fiche SAV (récupérer depuis JIRA), : ou les différents champs ont été testé : statuts, nom, prénom tel 1, adresse mail.

J'ai utilisé le même ticket « test » pour tester la création d'une fiche SAV

< Retour  SER-79030

Ticket Test/pour Dev

 Associer un ticket 
 Ajouter un formulaire
  Créer  ...

 **GOULJIAR Sammy** a créé cette demande via **API**
[Afficher la demande sur le portail](#)

Recherche de fiches SAV

ID Fiche: SER-
 Nom: _____ Prénom: _____ Téléphone: _____

 Rechercher

Créer une nouvelle fiche SAV

Informations client
 Nom: Gouljjar
 Prénom: Sammy
 Téléphone 1: 0692823999

Code RAV: 30304367
 Désignation: LVTI 12C 6P 47DB D
 EAN: 3660767997288

localhost:3000

⚠ Cet email est déjà associé à un client. Voulez-vous récupérer les infos associées ?

OK Annuler

- **2 utilisateurs**, représentant les différents rôles de l'application (admin, agent)

POST http://localhost:5000/auth/insertLogin

Body

```
1 {
2   "email": "test@test.com",
3   "mot_passe": "test",
4   "prenom": "prenomtest",
5   "nom": "nomtest",
6   "role": "admin"
}
```

201 Created • 407 ms • 1.09 KB

Body

```
1 {
2   "message": "Utilisateur créé avec succès",
3   "user": {
4     "id": 9,
5     "email": "test@test.com",
6     "prenom": "prenomtest",
7     "nom": "nomtest",
8     "role": "admin"
9   }
10 }
```

POST http://localhost:5000/auth/insertLogin

Body

```
1 {
2   "email": "agent@test.com",
3   "mot_passe": "test",
4   "nom": "nomtest",
5   "role": "agent"
6 }
7 }
```

400 Bad Request • 47 ms • 1022 B

Body

```
1 {
2   "message": "\"prenom\" is required"
3 }
```

POST ▼ http://localhost:5000/auth/insertLogin

Docs Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```

1 {
2   "email": "sammy.gouljiair@ravate.com",
3   "mot_passe": "diastest",
4   "prenom": "testnom",
5   "nom": "testprenom",
6   "role": "admin"
7 }

```

Body Cookies Headers (20) Test Results 🔄 **409 Conflict** • 338 ms • 102

HTML ▶ Preview 🐞 Debug with AI ▼

```

1 {"message": "Cet email existe déjà"}

```

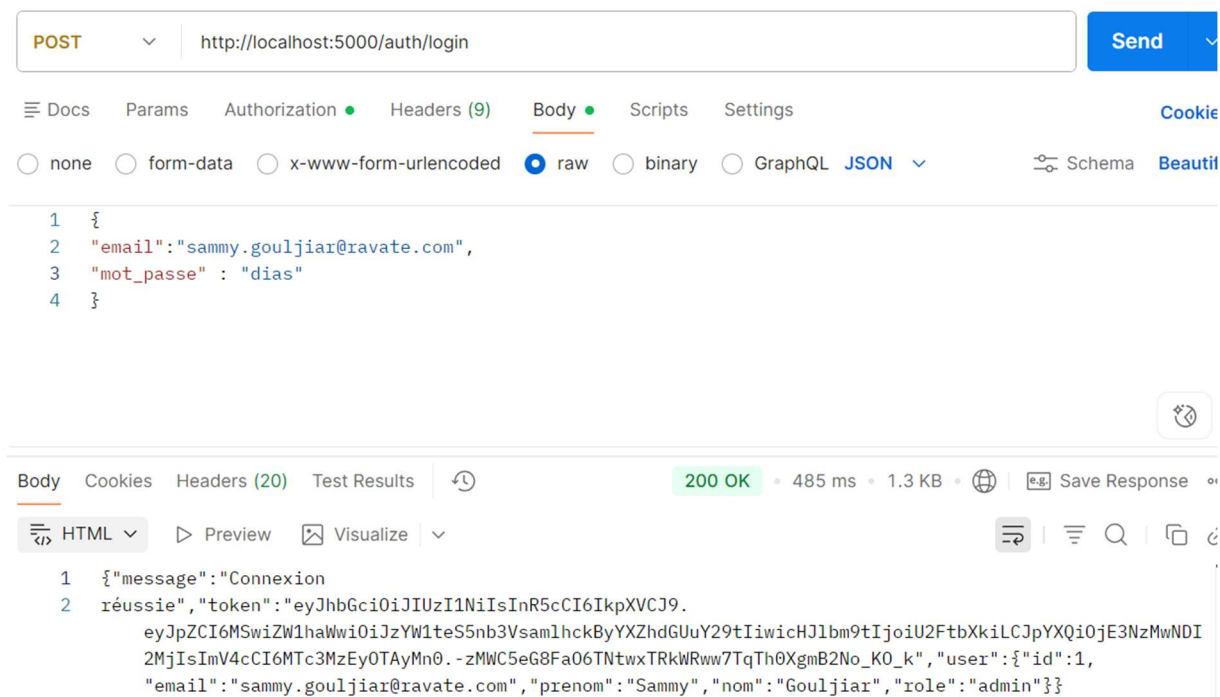
Nb : Le champ *email* de la table users est également sécurisé au niveau de la base de données grâce à une contrainte UNIQUE NOT NULL, assurant à la fois l'unicité des adresses email et empêchant l'insertion de valeurs nulles.

	A-Z Field	A-Z Type	A-Z Null	A-Z Key	A-Z Default	A-Z Extra
1	id	int(11)	NO	PRI	[NULL]	auto_increment
2	email	varchar(100)	NO	UNI	[NULL]	

Ces données ont permis de réaliser plusieurs scénarios de test afin de vérifier :

- le respect des règles métier définies dans l'application
- les contraintes d'unicité mises en place dans la base de données
- la gestion des erreurs applicatives
- la sécurisation des routes protégées nécessitant une authentification

- Des tests complémentaires ont également été effectués à l'aide de l'outil Postman, permettant de vérifier le bon fonctionnement des différentes routes de l'API ainsi que les codes de réponse HTTP retournés par le serveur, notamment :
- 200 : requête traitée avec succès



- 401 : accès non autorisé (utilisateur non authentifié)

```
router.get('/regroupement/statut', verifyToken, FicheController.getRegroupementByStatus);
```

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:5000/api/fiche/regroupement/statut
- Headers:** 9 hidden
- Body:** JSON
- Response:** 401 Unauthorized (32 ms, 1015 B)
- Response Body (JSON):**

```
{
  "message": "Token manquant"
}
```

- Ces tests ont permis de valider la robustesse de l'API et la conformité du comportement de l'application face aux différents scénarios d'utilisation.

12. CONCLUSION

Ce projet de digitalisation du Service Après-Vente (SAV) a permis de moderniser un processus métier essentiel au sein du Groupe Ravate.

Il s'inscrit dans une démarche globale d'amélioration de la performance opérationnelle et de fiabilisation des données.

Grâce à cette application, le SAV bénéficie désormais d'un outil moderne et centralisé qui a permis de :

- Automatiser les opérations répétitives, notamment la création, le suivi et la mise à jour des fiches SAV.
- Améliorer la traçabilité, la fiabilité et la cohérence des données tout au long du cycle de vie des produits.

- Offrir une interface intuitive, évolutive et adaptée aux besoins réels des utilisateurs (techniciens, responsables et fournisseurs).
- Sécuriser l'accès et la gestion des données grâce à un système d'authentification par jeton (JWT) et au chiffrement des mots de passe (bcryptjs).

Ce projet constitue aujourd'hui une base solide pour une intégration future avec Sage X3, permettant une synchronisation automatique des informations entre la gestion commerciale et le SAV.

az

TABLE DES MATIERES

1. INTRODUCTION – CONTEXTE DU PROJET	3
2. LISTE DES COMPÉTENCES COUVERTES PAR LE PROJET	4
3. RÉSUMÉ DU PROJET.....	6
4. SUIVI DU PROJET.....	7
A. Organisation et cadrage	7
B. Réalisation des maquettes	10
C. Développement back-end	11
C1 : Connexion à la base de donnée	13
C2. Connexion à l'API JIRA.....	14
C3. Services.....	16
C4. Contrôleurs.....	17
C4. Routes	18
C5. Middleware.....	19
D. Développement front-end.....	21
E. Versionning et gestion de code (GIT / GitHub)	22
5. CAHIER DES CHARGES	23
A. Besoins identifiés	23
B. Objectifs.....	23
C. Cibles et utilisateurs.....	23
D. Users Stories.....	24
E. MVP (Produit Minimum Viable).....	30
F. Évolutions potentielles	30
6. WIREFRAMES ET MAQUETTES	31
7. SPÉCIFICATIONS TECHNIQUES.....	36
A. Technologies utilisées	36
B. Navigateurs compatibles	37
C. Déploiement envisagé.....	37
D. Base de données.....	37
E. Routes	41
F. API JIRA	41
8. ARCHITECTURE	42
9. SÉCURITÉ ET VEILLE TECHNOLOGIQUE	43
10. LIVRABLE.....	45
11. JEUX d'ESSAI	46
12. CONCLUSION.....	50